



Zusammenfassung der Konzepte

- **Exception** Eine Exception ist ein Objekt, das Informationen über einen Programmfehler hält. Eine Exception wird ausgelöst, um zu signalisieren, dass ein Fehler aufgetreten ist.
- **Ungeprüfte Exception** Ungeprüfte Exceptions sind Exception-Typen, bei deren Verwendung der Compiler keine zusätzlichen Überprüfungen vornimmt.
- **Geprüfte Exception** Geprüfte Exceptions sind Exception-Typen, bei deren Verwendung der Compiler zusätzliche Überprüfungen durchführt und einfordert. Insbesondere erfordern geprüfte Exceptions in Java, dass **throws**-Klauseln und **try**-Blöcke angegeben werden müssen.
- **Exception-Handler** Ein Programmabschnitt, der Anweisungen schützt, in denen eine Exception ausgelöst werden kann, wird Exception-Handler genannt. Er definiert Anweisungen zur Meldung und/oder zum Wiederaufsetzen nach einer aufgetretenen Exception.
- **Zusicherung** Eine Tatsachenaussage, die bei normaler Programmausführung zutreffen sollte. Zu verwenden, um Annahmen zu formulieren und Programmierfehler leichter zu entdecken.
- **Serialisierung** Serialisierung erlaubt, ganze Objekte und Objekthierarchien in einer einzigen Operation zu lesen und zu schreiben. Jedes beteiligte Objekt muss dazu Instanz einer Klasse sein, die das Interface **Serializable** implementiert.

Übung 14.54 Ergänzen Sie das Projekt *Adressbuch* um die Fähigkeit, mehrere E-Mail-Adressen in einem **Kontakt**-Objekt zu speichern. Alle diese E-Mail-Adressen sollten gültige Schlüssel sein. Verwenden Sie auf jeder Stufe dieses Prozesses Zusicherungen und JUnit-Tests, um maximales Vertrauen in die endgültige Version zu schaffen.

KAPITEL

15

Entwurf von Anwendungen



Lernziele

Zentrale Konzepte in diesem Kapitel: Identifizieren von Klassen, Identifizieren von Klassen, CRC-Karten, Entwurfsmuster

Java-Konstrukte in diesem Kapitel: (In diesem Kapitel werden keine neuen Java-Konstrukte vorgestellt.)

In den vorangegangenen Kapiteln haben wir beschrieben, wie man gute Klassen schreibt. Wir haben diskutiert, wie man sie entwirft, wie man sie wartbar und robust macht und wie man sie miteinander interagieren lässt. All das ist wichtig, aber etwas sehr Wichtiges fehlt noch: das Finden der geeigneten Klassen.

In allen vorangegangenen Beispielen haben wir angenommen, dass wir mehr oder weniger wissen, welches die richtigen Klassen für unsere Problemlösungen sind. In einem realen Softwareprojekt kann das Finden der richtigen Klassen jedoch eines der schwierigsten Probleme sein. Diesen Aspekt der Softwareentwicklung wollen wir in diesem Kapitel näher betrachten.

Die frühen Phasen bei der Entwicklung eines Softwaresystems werden im Allgemeinen *Analyse und Entwurf* genannt. Wir analysieren das Problem, dann entwerfen wir eine Lösung. Der erste Entwurf wird dabei meist auf einer allgemeineren Ebene formuliert, als wir es in Kapitel 8 diskutiert haben. Wir werden zuerst überlegen, welche Klassen zur Problemlösung zu entwerfen sind und wie diese interagieren sollen. Sobald wir diesen Schritt abgeschlossen haben, können wir uns dem Entwurf der einzelnen Klassen und ihren Implementierungen zuwenden.

15.1 Analyse und Entwurf

Analyse und Entwurf von Softwaresystemen sind umfangreiche und komplexe Themengebiete. Ihre detaillierte Diskussion würde den Rahmen dieses Buches sprengen. Zahlreiche Methodiken sind in der Literatur beschrieben und werden in der Praxis eingesetzt. In diesem Kapitel wollen wir lediglich versuchen, einen Eindruck der mit diesen Tätigkeiten verbundenen Probleme zu geben.

Wir werden dazu eine sehr einfache Methode verwenden, die ihren Zweck für vergleichsweise kleine Probleme gut erfüllt. Um die ersten Klassen zu finden, werden wir die *Verb/Substantiv-Methode* verwenden. Anschließend werden wir *CRC-Karten* benutzen, um einen ersten Entwurf vorzunehmen.

15.1.1 Die Verb/Substantiv-Methode

Konzept

Bei der **Verb/Substantiv-Methode** korrespondieren Klassen in einem System in etwa mit den Substantiven in der Systembeschreibung. Die Methoden korrespondieren mit den Verben.

Bei dieser Methode geht es ausschließlich darum, die Klassen und Objekte für einen Problembereich sowie ihre Verknüpfungen und Interaktionen zu identifizieren. Die Substantive in der natürlichen Sprache beschreiben üblicherweise Dinge wie Menschen, Gebäude und so weiter. Die Verben beschreiben Aktionen wie schreiben, essen und so weiter. Aus diesen Konzepten der natürlichen Sprache können wir darauf schließen, dass bei einem Programmierproblem die Substantive oft mit den Klassen und Objekten korrespondieren, während die Verben oft mit den Dingen korrespondieren, die diese Objekte tun: also ihren Methoden. Wir brauchen keine sehr lange Beschreibung, um diese Technik zu illustrieren. Es reicht üblicherweise eine Darstellung in ein paar Absätzen.

Das Beispiel, an dem wir diesen Entwurfprozess verdeutlichen wollen, ist ein Buchungssystem für ein Kino.

15.1.2 Das Beispiel: ein Kinobuchungssystem

Diesmal werden wir nicht mit einem vorgegebenen System beginnen. Wir nehmen an, dass wir uns in einer Situation befinden, in der wir eine Anwendung von Grund auf neu entwerfen müssen. Die Aufgabe ist, für ein Kinounternehmen ein System zu entwickeln, mit dem die Plätze für Kinovorstellungen gebucht werden können. Kinobesucher rufen häufig im Voraus an, um Plätze zu reservieren. Die Anwendung sollte dann in der Lage sein, die freien Plätze zu finden und für den Besucher zu reservieren.

Wir nehmen an, dass wir schon einige Treffen mit den Betreibern des Kinounternehmens hatten, bei denen sie uns die Funktionalität beschrieben haben, die sie von dem System erwarten. (Die erwartete Funktionalität zu verstehen, sie zu beschreiben und diese Beschreibung mit dem Kunden abzustimmen, ist üblicherweise bereits ein Problem für sich. Diese Thematik geht jedoch weit über den Anspruch dieses Buches hinaus und kann in anderen Kursen und Büchern näher untersucht werden.)

Hier ist die Beschreibung, die wir für das Kinobuchungssystem formuliert haben:

Das Kinobuchungssystem sollte Platzreservierungen für mehrere Kinosäle verwalten. Jeder Kinosaal hat Plätze, die in Reihen angeordnet sind. Kinobesucher können Plätze reservieren und bekommen eine Reihen- und eine Platznummer zugewiesen. Sie können nach nebeneinanderliegenden Plätzen fragen.

Jede Platzreservierung gilt für eine bestimmte Vorstellung (also einen bestimmten Film zu einem bestimmten Zeitpunkt). Vorstellungen finden zu festgelegten Zeitpunkten an festgelegten Tagen statt und sind einem Kinosaal zugeteilt, in dem sie gezeigt werden. Das System speichert die Telefonnummer des Kinobesuchers.

Anhand dieses mehr oder weniger klar formulierten Textes können wir einen ersten Versuch vornehmen, auf Basis der Substantive und Verben Klassen und Methoden zu identifizieren.

15.1.3 Identifizieren von Klassen

Der erste Schritt beim Finden der Klassen besteht darin, dass wir durch die Beschreibung gehen und alle Substantive und Verben im Text markieren. Wenn wir dies tun, finden wir die folgenden Substantive und Verben (die Substantive sind in der Reihenfolge ihres Auftretens aufgeführt, die Verben sind den Substantiven zugeordnet, auf die sie sich beziehen):

Substantive:	Verben:
Kinobuchungssystem	<i>verwaltet</i> (Platzreservierungen) <i>speichert</i> (Telefonnummer)
Platzreservierung	<i>hat</i> (Plätze)
Kinosaal	
Platz	
Reihe	
Kinobesucher	<i>reserviert</i> (Plätze) <i>bekommt zugewiesen</i> (Reihennummer, Platznummer) <i>fragt nach</i> (nebeneinanderliegenden Plätzen)
Reihennummer	
Platznummer	
Vorstellung	<i>wird zugeteilt</i> (einem Kinosaal)
Film	
Zeitpunkt	
Tag	
Telefonnummer	

Die Substantive, die wir gefunden haben, geben uns einen ersten Hinweis auf die Klassen in unserer Anwendung. Als erste Näherung können wir eine Klasse für jedes Substantiv formulieren. Das ist kein exaktes Vorgehen – wir können später feststellen, dass wir noch zusätzliche Klassen benötigen oder einige der Substantive nicht benötigt werden. Das werden wir jedoch erst später herausfinden. Es ist auf jeden Fall wichtig, keine Substantive von Anfang an auszusortieren – wir haben noch nicht genügend Informationen, um eine fundierte Entscheidung treffen zu können.

Wenn wir diese Übung mit unseren Studenten machen, lassen einige Studenten sofort einige der Substantive weg. Zum Beispiel verzichtet einer den Studenten auf das Substantiv *Reihe* aus der obigen Beschreibung, weil es seiner Meinung nach nur eine Zahl ist, für das ein **int** reicht und keine Klasse benötigt wird. Auf dieser Stufe ist es wirklich wichtig, nicht den gleichen Fehler zu machen. Wir haben zu diesem Zeitpunkt einfach noch nicht genug Informationen, um zu entscheiden, ob *Reihe* ein **int** oder eine Klasse sein sollte. Diese Entscheidung können wir erst viel später treffen. Zurzeit gehen wir einfach nur mechanisch die Absätze durch und schreiben *alle* Substantive auf. Wir entscheiden noch nicht, welche Substantive „gut“ sind und welche nicht.

Sie haben möglicherweise festgestellt, dass alle Substantive im Singular aufgeführt sind. Es ist bei Klassen typisch, dass ihre Namen im Singular und nicht im Plural formuliert sind. Beispielsweise würden wir eine Klasse immer eher **Kino** als **Kinos** nennen, denn die Vervielfachung geschieht über das Erzeugen von Instanzen einer Klasse.

Übung 15.1 Denken Sie an die Projekte aus den vorherigen Kapiteln zurück. Erinnern Sie sich an Fälle, in denen ein Klassenname im Plural definiert war? Falls ja: Gab es in diesen Fällen gute Gründe für diese Entscheidung?

15.1.4 CRC-Karten

Der nächste Schritt in unserem Entwurfsprozess besteht darin, die Interaktionen zwischen unseren Klassen herauszuarbeiten. Dazu verwenden wir *CRC-Karten*.¹

CRC steht für Class/Responsibilities/Collaborators (Klasse/Zuständigkeiten/Partnerklassen). Die Idee ist, für jede Klasse eine Pappkarte (normale Karteikarten sind gut geeignet) zu verwenden. Wichtig ist, dass tatsächlich für jede Klasse eine eigenständige, anfassbare Karte angelegt wird und nicht eine Computerdarstellung oder ein einzelnes Blatt Papier für alle. Jede Karte wird in drei Bereiche unterteilt: ein Bereich oben links, in den der Name der Klasse geschrieben wird; ein Bereich darunter, in dem die Zuständigkeiten der Klasse eingetragen werden; und ein Bereich rechts, in dem die Partnerklassen eingetragen werden (Klassen, mit denen diese Klasse kooperiert). Abbildung 15.1 illustriert das Layout einer CRC-Karte.

Übung 15.2 Erstellen Sie CRC-Karten für die Klassen im Kinobuchungssystem. An dieser Stelle müssen Sie lediglich die Klassennamen eintragen.

¹ CRC-Karten wurden erstmals in dem Artikel „A Laboratory For Teaching Object-Oriented Thinking“ von Kent Beck und Ward Cunningham erwähnt. Dieser Artikel ist als Ergänzung zu diesem Kapitel lesenswert. Sie können ihn online unter <http://c2.com/doc/oopsla89/paper.html> finden oder eine Websuche nach dem Titel vornehmen.

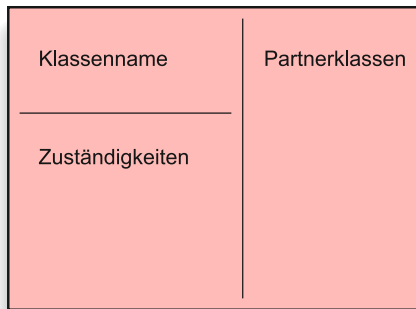


Abbildung 15.1
Aufbau einer
CRC-Karte.

15.1.5 Szenarios

Wir haben nun eine erste Näherung für die Klassen in unserem System und eine physische Repräsentation auf CRC-Karten. Um nun die notwendigen Interaktionen zwischen den Klassen in unserem System herauszufinden, spielen wir einige Szenarios durch. Ein Szenario ist ein Beispiel für eine Aktivität, die das System ausführen oder unterstützen muss. Szenarios werden häufig auch als Geschäftsfälle (*use case*) bezeichnet. Wir benutzen diesen Begriff hier nicht, da er häufig eine formale Art zur Beschreibung von Szenarios bezeichnet.

Szenarios lassen sich am besten in einer Gruppe durchspielen. Jedes Gruppenmitglied bekommt eine Klasse (oder eine kleine Anzahl an Klassen) zugewiesen und spielt die Rolle dieser Klasse, indem es laut ausspricht, was diese Klasse gerade tut. Während ein Szenario durchgespielt wird, halten die Teilnehmer auf den CRC-Karten alles fest, was die gerade agierende Klasse betrifft: wofür sie zuständig sein soll und mit welchen Klassen sie kooperieren muss.

Wir beginnen mit einem simplen Beispielszenario:

Ein Kunde ruft beim Kino an und möchte zwei Plätze für *Die Verurteilten* heute Abend buchen. Der Kinoangestellte versucht nun mit dem Buchungssystem, zwei Plätze zu finden und zu reservieren.

Da ein menschlicher Benutzer mit dem System interagiert (das durch die Klasse **Kinobuchungssystem** repräsentiert wird), fangen wir hier mit dem Szenario an. Als Nächstes könnte Folgendes passieren:

- Der Benutzer (der Kinoangestellte) möchte alle Vorstellungen von *Die Verurteilten* finden, die heute angesetzt sind. Also können wir auf der CRC-Karte **Kinobuchungssystem** als Zuständigkeit eintragen: *kann Vorstellungen nach Filmtitel und Tag finden*. Wir können auch die Klasse **Vorstellung** als Partnerklasse eintragen.
- Wir müssen uns fragen: Wie findet das System die Vorstellung? Wen fragt es dazu? Eine Lösung könnte sein, dass das **Kinobuchungssystem** eine Sammlung von Vorstellungen hält. Das führt uns zu einer weiteren Klasse: der Sammlung. (Diese könnte später durch eine **ArrayList**, eine **LinkedList**, ein **HashSet** oder eine andere Sammlung implementiert werden, mit zusätzlichen Methoden, die zu einer Vorstellung passen. Diese Entscheidung können wir später treffen – momentan notieren wir nur eine **VorstellungSammlung**.) Dies ist ein Beispiel dafür, dass wir beim Durchspielen von Szenarios weitere Klassen entdecken können. Es kann immer

Konzept

Szenarios (auch als „Geschäftsfälle“ oder unter dem englischen Begriff *use cases* bekannt) können benutzt werden, um ein Verständnis von den Interaktionen in einem System zu bekommen.

wieder passieren, dass wir Klassen aus Implementierungsgründen hinzufügen müssen, die wir bis dahin übersehen haben. Wir fügen also bei den Zuständigkeiten auf der Karte für das **Kinobuchungssystem** ein: *speichert eine Sammlung von Vorstellungen*. Und **VorstellungSammlung** wird als Partnerklasse eingetragen.

Übung 15.3 Erstellen Sie eine CRC-Karte für die neu erkannte Klasse **VorstellungSammlung** und fügen Sie diese in Ihr System ein.

- Wir nehmen an, dass es heute drei Vorstellungen gibt: eine um 17:30 Uhr, eine um 21:00 Uhr und eine um 23:30 Uhr. Der Angestellte teilt dem Kunden diese Zeiten mit und der Kunde wählt die Vorstellung um 21:00 Uhr. Der Angestellte möchte deshalb als Nächstes Informationen zu dieser Vorstellung abrufen (ob sie ausverkauft ist, in welchem Kinosaal sie läuft etc.). Also muss unser System in der Lage sein, die Details einer Vorstellung abzurufen und anzuzeigen. Spielen Sie dies durch. Die Person, die das Buchungssystem spielt, sollte die Person, die die Vorstellungen repräsentiert, nach den benötigten Details fragen. Daraufhin notieren Sie auf der Karte für das **Kinobuchungssystem**: *Ruft Details einer Vorstellung ab* und zeigt sie an, und auf der Karte Vorstellung: *Liefert Informationen über Kinosaal und Anzahl der freien Plätze*.
- Nehmen wir an, dass es reichlich freie Plätze gibt. Der Kunde wählt die Plätze 13 und 14 in Reihe 12. Der Angestellte nimmt die Buchung vor. Wir notieren auf der Karte **Kinobuchungssystem**: *Akzeptiert Platzreservierungen vom Benutzer*.
- Wir müssen nun durchspielen, wie eine Platzreservierung ablaufen soll. Eine Platzreservierung ist sicherlich an eine Vorstellung gebunden. Das **Kinobuchungssystem** sollte also die Vorstellung über die Reservierung informieren. Es delegiert das tatsächliche Durchführen der Reservierung an das Objekt Vorstellung. Wir können also für Vorstellung notieren: *Kann Plätze reservieren*. (Sie haben vielleicht schon festgestellt, dass die Grenze zwischen Klasse und Objekt sehr leicht verschwimmt, wenn man mit CRC-Karten umgeht; letztlich repräsentiert die Person, die eine Klasse repräsentiert, auch die Instanzen dieser Klasse. Das ist absichtlich so und im Allgemeinen auch kein Problem.)
- Jetzt sind wir bei der Klasse **Vorstellung**. Sie hat die Anfrage bekommen, einen Platz zu reservieren. Wie geht sie mit dieser Anfrage um? Um Platzreservierungen vornehmen zu können, muss sie auf eine Repräsentation der Plätze in einem Kinosaal zugreifen können. Also nehmen wir an, dass eine Vorstellung eine Verbindung zu einem **Kinosaal**-Objekt hat. (Notieren Sie dies auf der Karte: *Speichert Kinosaal*. Diese Klasse wird damit auch zur Partnerklasse.) Der Kinosaal sollte vermutlich die exakte Anzahl und Anordnung seiner Plätze kennen. (Wir können an dieser Stelle im Hinterkopf behalten oder auf einem Blatt Papier notieren, dass jede Vorstellung eine eigene Kopie eines Kinosaals haben sollte, damit eine Reservierung für eine Vorstellung nicht auch einen Platz in einer anderen Vorstellung blockiert. Darauf sollten wir achten, wenn eine **Vorstellung** erzeugt wird. Wir werden darüber später noch einmal nachdenken, wenn wir ein anderes Szenario durchspielen: *das Ansetzen neuer Vor-*

stellungen.) Vermutlich wird also eine Vorstellung die Reservierung wiederum an einen Kinosaal delegieren.

- Der Kinosaal hat nun die Anfrage bekommen, einen Platz zu reservieren. (Notieren Sie dies auf der Karte: *Akzeptiert Reservierungsanfrage*.) Wie geht er damit um? Der Kinosaal könnte eine Sammlung seiner Plätze halten. Oder er könnte eine Sammlung von Reihen halten (und jede Reihe ist ein eigenes Objekt), und Reihen wiederum halten Plätze. Welche dieser Alternativen ist besser? Wenn wir bereits einige andere Szenarios im Auge haben, dann entscheiden wir uns vermutlich für den Ansatz mit den Reihen. Wenn ein Kunde beispielsweise vier Plätze nebeneinander in derselben Reihe reservieren möchte, dann könnten benachbarte Plätze einfacher zu finden sein, wenn sie alle in einer Reihe organisiert sind. Wir notieren also auf der Karte **Kinosaal**: *Speichert Reihen*. Und **Reihe** wird damit zur Partnerklasse.
- Wir notieren auf der Karte **Reihe**: *Speichert Sammlung von Plätzen*. Und als neue Partnerklasse: **Platz**.
- Zurück zur Klasse **Kinosaal**. Wir haben noch nicht entschieden, wie sie mit der Reservierungsanfrage umgehen soll. Nehmen wir an, dass sie zwei Dinge tut: Sie findet die gewünschte Reihe und stellt an das **Reihe**-Objekt die Anfrage, die gewünschten Plätze zu reservieren.
- Als Nächstes notieren wir auf der Karte **Reihe**: *Akzeptiert Reservierungsanfrage für einen Platz*. Es muss dazu das entsprechende **Platz**-Objekt finden (wir können als Zuständigkeit notieren: *Kann Plätze über ihre Nummer finden*) und kann eine Reservierung dieses Platzes vornehmen. Es wird dies tun, indem es dem **Platz**-Objekt mitteilt, dass es nun reserviert ist.
- Wir können nun auf der Karte **Platz** notieren: *Akzeptiert Reservierungen*. Der Platz selbst kann sich merken, dass er reserviert worden ist. Wir notieren auf der Karte **Platz**: *Speichert Reservierungszustand (freilreserviert)*.

Übung 15.4 Spielen Sie dieses Szenario auf Ihren Karten durch (mit einer Gruppe von Personen, falls möglich). Fügen Sie weitere Informationen hinzu, die Ihrer Meinung nach bisher ausgelassen wurden.

Sollte ein Platz auch speichern, wer ihn reserviert hat? Er könnte den Namen des Kunden oder seine Telefonnummer speichern. Oder vielleicht sollten wir ein **Kunden**-Objekt erzeugen, sobald jemand eine Reservierung vornimmt, und eine Referenz auf dieses **Kunden**-Objekt im reservierten Platz speichern? Das sind spannende Fragen und wir werden versuchen, die beste Antwort über das Durchspielen von Szenarios zu finden.

Dies war nur ein erstes, einfaches Szenario. Wir müssen noch viele weitere durchspielen, bevor wir ein angemessenes Verständnis vom gewünschten System bekommen.

Das Durchspielen von Szenarios funktioniert am besten, wenn eine Gruppe von Personen um einen Tisch herumsitzt und die Karten nach Bedarf verschoben

werden. Karten, die eng zusammenarbeiten, können auch räumlich enger angeordnet werden, um die Kopplung im System zu visualisieren.

Weitere durchzuspielende Szenarios würden unter anderem die folgenden sein:

- Ein Kunde möchte fünf Plätze nebeneinander reservieren. Spielen Sie möglichst exakt durch, wie diese fünf Plätze gefunden werden.
- Ein Kunde ruft an und teilt mit, dass er die Platznummern, die er gestern reservieren ließ, vergessen hat. Könnten Sie die Platznummern bitte noch einmal herausfinden?
- Ein Kunde ruft an und möchte seine Reservierung stornieren. Er kann seinen Namen und die Vorstellung angeben, aber er hat die Platznummern vergessen.
- Eine Kundin, die bereits reserviert hat, ruft an. Sie möchte wissen, ob sie einen weiteren Platz direkt neben ihren bereits reservierten Plätzen bekommen kann.
- Eine Vorstellung wird abgesagt. Das Kino möchte alle Kunden anrufen, die für diese Vorstellung reserviert haben.

Diese Szenarios sollten Ihnen ein gutes Verständnis von dem Teil des Systems vermitteln, das mit Platzsuche und -reservierung zu tun hat. Wir brauchen noch eine weitere Gruppe von Szenarios: solche, die mit dem Aufbau der Kinosäle und dem Ansetzen von Vorstellungen zu tun haben. Einige mögliche wären:

- Das System soll für ein neues Kino eingerichtet werden. Das Kino hat zwei Kinosäle verschiedener Größe. Saal A hat 26 Reihen mit jeweils 18 Plätzen. Saal B hat 32 Reihen, wobei die ersten sechs Reihen 20 Plätze haben, die nächsten 10 Reihen haben 22 Plätze und die restlichen Reihen 26 Plätze.
- Ein neuer Film kommt in die Kinos. Er wird in den nächsten zwei Wochen dreimal täglich gezeigt (um 16:40 Uhr, um 18:30 Uhr und um 20:30 Uhr). Die Vorstellungen müssen neu in das System eingefügt werden. Alle Vorstellungen laufen im Kinosaal A.

Übung 15.5 Spielen Sie diese Szenarios durch. Notieren Sie alle unbeantworteten Fragen auf einem getrennten Blatt Papier. Protokollieren Sie, welche Szenarios Sie bereits durchgespielt haben.

Übung 15.6 Welche weiteren Szenarios können Sie sich vorstellen? Schreiben Sie diese auf und spielen Sie sie durch.

Das Durchspielen von Szenarios erfordert einige Geduld und auch Übung. Sie sollten sich dafür genügend Zeit nehmen. Die hier genannten Szenarios können mehrere Stunden dauern.

Neulinge mit dieser Technik neigen häufig zu Abkürzungen, indem sie nicht jedes Detail während des Durchspielens hinterfragen oder nicht ausreichend festhalten. Das ist gefährlich! Wir werden sehr bald dazu übergehen, dieses System in Java zu implementieren, und wenn Details offengeblieben sind, dann besteht die Gefahr, dass bei der Implementierung Entscheidungen ad hoc getroffen werden, die sich später als schlecht erweisen könnten.

Neulinge neigen auch dazu, Szenarios zu vergessen. Wenn Teile nicht genügend durchdacht sind, bevor mit dem Entwurf und der Implementierung von Klassen begonnen wird, kann der Aufwand sehr groß werden, wenn bereits implementierte Teile wieder geändert werden müssen.

Die Szenarios gut durchzuspielen, sorgfältig alle notwendigen Schritte durchzugehen und die Schritte ausführlich festzuhalten, bedarf einiger Übung und einer großen Disziplin. Diese Aufgabe ist schwerer, als sie aussieht, und wichtiger, als Ihnen vielleicht zurzeit klar ist.

Übung 15.7 Machen Sie einen Entwurf für die Simulation eines Flughafenkontrollsystems. Verwenden Sie CRC-Karten und Szenarios. Hier ist die Beschreibung des Systems:

- *Das Programm ist eine Simulation eines Flughafens. Für einen neu zu bauenden Flughafen müssen wir herausfinden, ob wir mit zwei Rollfeldern auskommen oder ob wir drei benötigen. Der Flughafen funktioniert folgendermaßen:*
- *Es gibt mehrere Rollfelder. Flugzeuge starten von und landen auf diesen Rollfeldern. Fluglotsen kontrollieren den Verkehr und geben Flugzeugen die Erlaubnis, zu landen oder zu starten. Die Lotsen geben teilweise die Erlaubnis unmittelbar, teilweise lassen sie Flugzeuge warten. Flugzeuge müssen einen Mindestabstand zueinander einhalten. Der Zweck des Programms ist, den Flughafen im Betrieb zu simulieren.*

15.2 Klassenentwurf

Jetzt ist der nächste größere Schritt zu machen: von den CRC-Karten zu Java-Klassen. Während der Übungen mit den CRC-Karten sollten Sie ein gutes Verständnis von der Struktur Ihrer Anwendung bekommen haben und davon, wie die Klassen zur Erfüllung der Aufgaben des Systems zusammenarbeiten. Möglicherweise hatten Sie Situationen, in denen neue Klassen eingeführt werden mussten (das gilt meist für Klassen, die interne Datenstrukturen repräsentieren), und möglicherweise hatten Sie auch Karten für Klassen, die gar nicht benutzt wurden. Wenn das der Fall war, können Sie diese Karten nun entfernen.

Das Erkennen der Klassen für die Implementierung ist nun trivial: Die Karten zeigen uns den kompletten Satz an Klassen, den wir benötigen. Das Festlegen der Schnittstellen dieser Klassen (also ihrer öffentlichen Methoden) ist etwas schwieriger, aber auch dafür haben wir bereits einen wichtigen Schritt getan. Wenn das Durchspielen der Szenarios gut gelaufen ist, dann sollten die Zuständigkeiten, die für die Klassen notiert wurden, mehr oder weniger ihre öffentlichen Methoden beschreiben (und möglicherweise auch einige ihrer Datenfelder). Die Zuständigkeiten jeder Klasse sollten nach den Prinzipien aus Kapitel 8 ausgewertet werden: Entwurf nach Zuständigkeiten, Kopplung und Kohäsion.

15.2.1 Entwurf von Klassenschnittstellen

Bevor wir unsere Anwendung in Java implementieren, können wir die Karten noch ein weiteres Mal benutzen, um durch ein Übersetzen der informellen Beschreibungen in Methoden mit Parametern dem endgültigen Entwurf näher zu kommen.

Um zu einer formaleren Beschreibung zu kommen, können wir die Szenarios noch einmal durchspielen, diesmal aber unter Benutzung der Begriffe Methodenaufruf, Parameter und Ergebniswerte. Die Logik und der Ablauf der Anwendung sollten sich nicht mehr ändern; wir versuchen nun, die Methodensignaturen und Instanzfelder komplett zu definieren. Dies tun wir mit einem neuen Satz Karten.

Übung 15.8 Erstellen Sie einen neuen Satz CRC-Karten für die Klassen, die Sie identifiziert haben. Spielen Sie die Szenarios erneut durch. Notieren Sie diesmal exakte Methodennamen für alle Aufrufe anderer Klassen und geben Sie alle Details (Name und Typ) aller Parameter, die übergeben werden, sowie der Ergebniswerte an. Die Methodensignaturen werden anstelle der Zuständigkeiten auf die CRC-Karten geschrieben. Auf der Rückseite einer Karte können Sie die Datenfelder notieren, die eine Klasse haben soll.

Sobald wir die oben beschriebene Übung durchgeführt haben, ist das Erstellen der Klassenschnittstellen einfach. Wir können direkt von den Karten nach Java übersetzen. Üblicherweise sollten alle Klassen angelegt werden und für alle öffentlichen Methoden sollten sogenannte *Stubs* definiert werden. Ein Stub (englisch für Stumpf, Stummel) ist ein Platzhalter für eine Methode, der die korrekte Signatur, aber einen leeren Rumpf hat.²

Viele Studenten halten dieses Vorgehen für zu aufwendig. Am Ende des Projekts werden Sie aber hoffentlich den Wert dieser Aktivitäten zu schätzen wissen. Viele Entwicklungsteams haben nachträglich feststellen müssen, dass die Zeit, die sie während der Entwurfsphase eingespart haben, mehrfach aufgewendet werden musste, um Fehler oder Auslassungen zu beheben, die zu spät entdeckt wurden.

Unerfahrene Programmierer sehen das Schreiben von Quelltexten als das „eigentliche Programmieren“ an. Das Erstellen eines grundlegenden Entwurfs wird, wenn nicht als überflüssig, so doch meist als ärgerlich angesehen, und die meisten Entwickler können gar nicht früh genug mit der „echten“ Arbeit anfangen. Das ist eine gefährliche Sichtweise.

Der grundlegende Entwurf ist einer der wichtigsten Bestandteile eines Projekts. Sie sollten mindestens so viel Zeit für den Entwurf einplanen, wie Sie für die Implementierung vorgesehen haben. Anwendungsentwurf ist nicht etwas, das vor dem Programmieren durchgeführt wird – es *ist* (der wichtigste Teil vom) Programmieren.

² Bei Bedarf können Sie bei den Methoden, die einen anderen Ergebnistyp als **void** haben, triviale **return**-Anweisungen angeben. Liefern Sie als Ergebnis einfach den Wert **null** für Objekttypen und null oder **false** für die primitiven Typen.

Fehler in der Implementierung lassen sich später relativ leicht beheben. Fehler im Entwurf können im besten Fall teuer in der Korrektur sein, im schlimmsten Fall sind sie fatal für die Anwendung. In manchen Fällen sind sie praktisch gar nicht mehr zu beheben (und erfordern einen fast kompletten Neubeginn).

15.2.2 Entwurf von Benutzungsschnittstellen

Ein Aspekt, den wir bisher in der Diskussion nicht berücksichtigt haben, ist der Entwurf der Benutzungsschnittstelle.³ An einem bestimmten Punkt müssen wir detailliert festlegen, was ein Benutzer am Bildschirm zu sehen bekommt und wie er mit dem System interagieren kann.

In einer gut entworfenen Anwendung ist dieser Aspekt weitgehend unabhängig von der Funktionalität der Anwendung, sodass er unabhängig von den übrigen Klassen entworfen werden kann. In den vorangehenden Kapiteln haben wir gesehen, dass wir durch BlueJ die Mittel haben, mit der Anwendung zu interagieren, noch bevor eine endgültige Benutzungsschnittstelle zur Verfügung steht, und können somit die interne Struktur zuerst bearbeiten.

Die Benutzungsschnittstelle kann grafisch sein, mit Knöpfen und Menüs, oder sie kann textbasiert sein oder wir können ausschließlich mit dem Aufrufmechanismus von BlueJ arbeiten. Vielleicht läuft das System in einem Netzwerk und die Benutzungsschnittstelle wird auf verschiedenen Rechnern im Webbrowser angezeigt.

Wir werden vorläufig den Entwurf der Benutzungsschnittstelle ignorieren und mithilfe von BlueJ mit unserem Programm interagieren.

15.3 Dokumentation

Nachdem wir die Klassen und ihre Schnittstellen identifiziert haben und bevor wir die Methoden einer Klasse implementieren, sollten wir ihre Schnittstelle dokumentieren. Dies umfasst das Schreiben eines Klassenkommentars und der Methodenkommentare für alle Klassen im System. Diese sollten so ausführlich sein, dass der Zweck jeder Klasse und jeder Methode deutlich wird.

Neben der Analyse und dem Entwurf ist die Dokumentation ein weiterer Bereich, der von Neulingen leicht vernachlässigt wird. Es ist nicht leicht für einen unerfahrenen Programmierer zu erkennen, warum die Dokumentation so wichtig ist. Das liegt daran, dass unerfahrene Programmierer meist an Projekten arbeiten, die nur aus einer Handvoll Klassen bestehen und die sich meist nur über wenige Wochen oder Monate erstrecken. Ein Programmierer kann mit einer schlechten Dokumentation durchkommen, wenn er an solchen Miniprojekten arbeitet.

³ Beachten Sie die Doppelbedeutung von *Entwurf von Schnittstellen* an diesem Punkt! Vorher haben wir über die Schnittstellen von einzelnen Klassen gesprochen (den Satz ihrer öffentlichen Methoden); hier reden wir nun über die *Benutzungsschnittstelle* – den Teil, den ein Benutzer auf dem Bildschirm sieht und über den er mit der Anwendung interagieren kann. Beides sind wichtige Dinge und unglücklicherweise wird der Begriff *Schnittstelle* für beide verwendet.

Andererseits wundern sich selbst erfahrene Programmierer häufig, wie man eine Dokumentation noch vor der Implementierung schreiben kann. Eine gute Dokumentation ist auf einer abstrakteren Ebene gehalten, die lediglich beschreibt, was eine Klasse oder Methode tut, aber nicht die Details darstellt, wie dies geschieht. Dies ist normalerweise symptomatisch, wenn die Implementierung als wichtiger erachtet wird als der Entwurf.

Wenn ein Entwickler sich mit interessanteren Problemen beschäftigen will und professionell in realen Projekten arbeitet, dann arbeitet er nicht selten mit Dutzenden von Kollegen über mehrere Jahre zusammen. Die *Ad-hoc*-Lösung, die „Dokumentation im Kopf“ zu haben, funktioniert dann nicht mehr.

Übung 15.9 Erzeugen Sie ein BlueJ-Projekt für das Kinobuchungssystem. Erzeugen Sie die notwendigen Klassen. Legen Sie Stubs für alle Methoden an.

Übung 15.10 Dokumentieren Sie alle Klassen und Methoden. Wenn Sie in einer Gruppe gearbeitet haben, dann weisen Sie den Gruppenmitgliedern die Zuständigkeit für einzelne Klassen zu. Verwenden Sie das **javadoc**-Format für Ihre Kommentare mit den passenden **javadoc**-Schlüsselwörtern für die Details.

15.4 Kooperation

Programmieren im Paar

Traditionell werden Klassen von Einzelpersonen implementiert. Die meisten Programmierer entwickeln ihren Quelltext allein, andere Personen werden meist erst einbezogen, wenn die Implementierung abgeschlossen ist und bewertet oder getestet werden soll.

In den letzten Jahren wurde verstärkt das Programmieren im Paar (*pair programming*) als eine Technik empfohlen, mit deren Hilfe besserer Quelltext (besser strukturiert und mit weniger Fehlern) erstellt werden kann. Programmieren im Paar ist auch ein Element des *Extreme Programming*. Suchen Sie im Web nach den Begriffen „pair programming“ und „extreme programming“, um mehr zu erfahren.

Softwareentwicklung wird üblicherweise in Teams vorgenommen. Ein wohlstrukturierter objektorientierter Ansatz liefert eine gute Unterstützung für Teamarbeit, denn er erlaubt die Zerlegung einer Aufgabe in lose gekoppelte Komponenten (Klassen), die unabhängig voneinander implementiert werden können.

Obwohl die grundlegende Entwurfsarbeit im Team erfolgt ist, sollten Sie nun getrennt arbeiten. Wenn die Definition der Klassenschnittstellen und der Dokumentation gut gemacht wurde, sollten die Klassen unabhängig voneinander implemen-

tiert werden können. Klassen können also Programmierern zugewiesen werden, die diese allein oder im Paar umsetzen.

Die Implementierungsphase für das Kinobuchungssystem werden wir hier nicht im Detail besprechen. In dieser Phase kommen all die Techniken und Vorgehensweisen zum Einsatz, die wir in den vorherigen Kapiteln kennengelernt haben; wir hoffen also, dass die Leser inzwischen selbst wissen, wie sie von diesem Punkt aus weiter vorgehen müssen.

15.5 Prototyping

Anstatt eine Anwendung in einem einzigen, großen Schritt zu entwerfen und zu entwickeln, können *Prototypen* verwendet werden, um Teile eines Systems zu erkunden.

Ein Prototyp ist eine Version einer Anwendung, in der Teile dieser Anwendung simuliert werden, um mit anderen Teilen experimentieren zu können. Sie können beispielsweise einen Prototyp entwickeln, um eine grafische Benutzungsschnittstelle zu testen. Die Funktionalität der Anwendung ist dann eventuell nicht vollständig implementiert. Stattdessen implementieren Sie Methoden, die ihre Aufgaben nur simulieren. Beispielsweise könnte beim Aufruf der Methode des Kinosystems, die einen freien Platz suchen soll, immer *Platz 3, Reihe 15* geliefert werden, ohne dass wirklich eine Suche implementiert ist. Prototyping erlaubt uns, ein ausführbares (aber nicht voll funktionierendes) System schnell zu entwickeln, um Teile der Anwendung in der Praxis zu erproben.

Prototypen sind auch nützlich für einzelne Klassen, um die Entwicklung im Team zu unterstützen. Wenn verschiedene Teammitglieder an unterschiedlichen Klassen arbeiten, werden diese Klassen oft nicht gleichzeitig fertig. In einigen Fällen kann eine fehlende Klasse die Entwicklung und den Test anderer Klassen aufhalten. Dann kann es nützlich sein, für diese Klasse einen Prototyp zu schreiben. Der Prototyp implementiert alle Methoden mit Stubs; diese enthalten keine vollständig abgeschlossenen Implementierungen, sondern simulieren ihre Funktionalität nur. Dieser Prototyp sollte schnell erstellt werden, damit Entwickler von Klassen, die diese Klasse benutzen müssen, bis zur Fertigstellung der vollständigen Implementierung weiterarbeiten können.

In Abschnitt 15.6 werden wir auch ansprechen, dass ein Prototyp zusätzlich dazu dienen kann, Schwierigkeiten oder Probleme aufzudecken, die in einer frühen Phase noch nicht bedacht wurden.

Konzept

Prototyping ist die Konstruktion von unvollständigen Systemen, in denen einige Anteile simuliert werden. Es dient dazu, möglichst früh ein Verständnis von der Arbeitsweise eines Systems zu bekommen.

Übung 15.11 Umreißen Sie einen Prototyp für Ihr Kinosystem. Welche Klassen sollten zuerst implementiert werden und welche sollten vorläufig nur als Prototypen realisiert werden?

Übung 15.12 Implementieren Sie den Prototyp Ihres Kinosystems.

15.6 Softwarewachstum

Es gibt viele Modelle, nach denen Software entwickelt werden kann. Das bekannteste ist das sogenannte *Wasserfallmodell* (so benannt, weil Aktivitäten stufenweise nacheinander erfolgen wie die Zwischenstufen bei manchen Wasserfällen – es gibt üblicherweise kein Zurück).

15.6.1 Das Wasserfallmodell

Im Wasserfallmodell werden verschiedene Phasen der Softwareentwicklung in einer festen Reihenfolge vorgenommen:

- Analyse des Problems
- Entwurf der Software
- Implementierung der Softwarekomponenten
- Modultests
- Integrationstests
- Auslieferung des Systems an den Kunden

Wenn eine dieser Phasen fehlschlägt, müssen wir möglicherweise eine Stufe zurückgehen – wenn beispielsweise Tests fehlschlagen, müssen wir zurück zur Implementierung –, aber ein weiteres Zurückgehen ist nicht vorgesehen.

Dies ist vermutlich das etablierteste und konservativste Vorgehensmodell für Softwareentwicklung, das seit vielen Jahren weitverbreitet ist. Über die Jahre wurden jedoch auch etliche Nachteile dieses Modells festgestellt. Die beiden wichtigsten Schwächen sind die Annahmen, dass die Entwickler die Funktionalität des zu erstellenden Systems von Anfang an vollständig verstanden haben und ein System sich nach seiner Auslieferung nicht mehr ändert.

In der Praxis treffen beide Annahmen meist nicht zu. Es kommt häufig vor, dass der Entwurf der Systemfunktionalität nicht von Anfang an perfekt ist; meist liegt dies daran, dass der Kunde, der sein Aufgabengebiet sehr gut kennt, nicht viel von Softwareentwicklung versteht, und die Softwaretechniker, die gute Programme schreiben können, nur ein begrenztes Verständnis des Aufgabengebiets haben.

15.6.2 Iterative Vorgehensmodelle

Eine Möglichkeit, den Schwächen des Wasserfallmodells beizukommen, ist der frühe Einsatz von Prototypen und die Einbeziehung des Kunden in den Entwicklungsprozess. Es werden Prototypen entwickelt, die anfangs nur einen Eindruck davon vermitteln, wie sich das System präsentiert und was es tun wird, und der Kunde beurteilt regelmäßig den Entwurf und die Funktionalität. Dies führt zu einem eher zyklischen Vorgehen als beim Wasserfallmodell. Hier iteriert die Softwareentwicklung mehrfach durch einen Zyklus *Analyse/Entwurf/Prototyp – Implementierung – Feedback des Kunden*.

Ein anderer Ansatz besagt, dass Software nicht entworfen wird, sondern *wächst*. Die Idee ist, dass zu Anfang ein kleines und sauber entworfenes System gebaut wird, das der Endbenutzer wirklich einsetzen kann. Anschließend wird nach und nach in kontrollierter Weise weitere Funktionalität hinzugefügt (die Software

wächst), sodass mehrfach und in regelmäßigen Abständen komplett benutzbare und auslieferbare Softwarestände erstellt werden.

In der Realität steht ein solches Wachsen natürlich nicht im Widerspruch zum Entwurf von Software. Jeder Wachstumsschritt wird sorgfältig entworfen. Aber es wird nicht versucht, das System von Anfang an komplett zu entwerfen. Denn: Eigentlich existiert so etwas wie ein fertiges Softwaresystem gar nicht!

Das traditionelle Wasserfallmodell hat ein vollständiges Softwaresystem zum Ziel. Das Modell vom Softwarewachstum hingegen geht davon aus, dass es vollständige Systeme, die auf lange Sicht unverändert benutzt werden, nicht gibt. Es gibt nur zwei Dinge, die mit einer Software geschehen können: Entweder wird sie ständig angepasst und verbessert oder sie wird bedeutungslos.

Diese Diskussion ist von großer Bedeutung für dieses Buch, denn sie beeinflusst unsere Sicht in Bezug darauf, welche Aufgaben und Fähigkeiten wir für einen Programmierer oder Softwaretechniker für wichtig halten. Möglicherweise haben Sie schon erraten, dass die Autoren dieses Buches das Modell der iterativen Entwicklung dem Wasserfallmodell deutlich vorziehen.⁴

Deshalb bekommen bestimmte Aufgaben und Fähigkeiten einen sehr viel höheren Stellenwert, als sie im Wasserfallmodell haben würden: Softwarewartung, Quelltexte lesen (und nicht nur schreiben), Entwurf für Erweiterbarkeit, Dokumentation, Erstellung von lesbaren Quelltexten und viele weitere Aspekte, die wir in diesem Buch erwähnt haben, bekommen ihre Relevanz durch die Tatsache, dass wir genau wissen, dass nach uns andere kommen werden, die unsere Quelltexte anpassen und erweitern müssen.

Ein Stück Software als eine ständig wachsende, sich ändernde und sich anpassende Einheit zu betrachten und nicht als ein statisches Stück Text, das geschrieben und erhalten wird wie ein Roman, bestimmt unsere Vorstellung davon, wie gute Software erstellt werden sollte. Alle Techniken, die wir in diesem Buch behandeln, zielen darauf ab.

Übung 15.13 Auf welche Weise könnte das Kinobuchungssystem in Zukunft angepasst oder erweitert werden? Welche Änderungen sind wahrscheinlicher als andere? Erstellen Sie eine Liste der möglichen zukünftigen Änderungen.

Übung 15.14 Gibt es andere Organisationen, die ähnliche Buchungssysteme wie das beschriebene verwenden könnten? Welche Unterschiede bestehen zwischen diesen Systemen?

Übung 15.15 Können Sie sich den Entwurf eines „generischen“ Buchungssystems vorstellen, das jeweils für die Zwecke verschiedener Organisationen spezifisch angepasst und zugeschnitten wird? Wenn Sie ein solches System entwerfen sollten, an welchem Punkt im Entwicklungsprozess des Kinosystems würden Sie Änderungen vornehmen? Oder würden Sie dieses System ignorieren und von Grund auf neu beginnen?

⁴ Ein hervorragendes Buch über die Probleme bei der Softwareentwicklung und ihre möglichen Lösungen ist *The Mythical Man-Month* von Frederick P. Brooks, erschienen bei Addison-Wesley. Obwohl die erste Ausgabe inzwischen 40 Jahre alt ist, ist es nach wie vor eines der spannendsten und lehrreichsten Bücher zu diesem Thema.

15.7 Der Einsatz von Entwurfsmustern

In früheren Kapiteln haben wir ausführlich Techniken vorgestellt, mit denen wir Teile unserer Arbeit wiederverwenden und durch die wir unsere Quelltexte für andere besser verständlich machen können. Diese Diskussionen sind jedoch meist auf der Ebene des Quelltextes einzelner Klassen geblieben.

Je erfahrener wir werden und je größer die Systeme werden, die wir bauen, desto mehr treten die Schwierigkeiten bei der Implementierung einzelner Klassen in den Hintergrund. Die Struktur des Gesamtsystems hingegen – die komplexen Beziehungen zwischen den Klassen – wird zunehmend schwieriger zu entwerfen und zu verstehen als der Quelltext einzelner Klassen.

Es ist konsequent, wenn wir versuchen, für Klassenstrukturen dieselben Ziele zu erreichen wie für Quelltexte: Wir wollen möglichst große Teile wiederverwenden und wir wollen, dass andere unsere Strukturen verstehen können.

Konzept

Ein **Entwurfsmuster** ist eine Beschreibung eines Entwurfsproblems, verbunden mit der Beschreibung einer kleinen Menge an Klassen und ihrer Interaktionsstruktur, mit deren Hilfe das Problem gelöst werden kann.

Auf der Ebene von Klassenstrukturen können beide Ziele durch die Verwendung von *Entwurfsmustern* unterstützt werden. Ein Entwurfsmuster beschreibt ein bei der Softwareentwicklung häufig auftretendes Problem und liefert dann eine allgemeine Lösung für dieses Problem, die in vielen Zusammenhängen eingesetzt werden kann. Bei einem Softwareentwurfsmuster besteht diese Lösung meist aus der Beschreibung einer kleinen Menge von Klassen mitsamt ihren Interaktionen.

Entwurfsmuster helfen uns auf zweierlei Weise. Erstens dokumentieren sie gute Lösungen zu Problemen, sodass diese Lösungen später für ähnliche Probleme benutzt werden können. Die Wiederverwendung findet also nicht auf Ebene des Quelltextes statt, sondern auf der Ebene der Klassenstruktur.

Zweitens haben Entwurfsmuster Namen und bilden auf diese Weise ein Vokabular, mit dessen Hilfe Softwaretechniker über ihre Entwürfe reden können. Wenn erfahrene Entwickler die Struktur einer Anwendung diskutieren, dann könnte einer sagen: „An dieser Stelle sollten wir ein Singleton verwenden.“ *Singleton* ist der Name eines bekannten Entwurfsmusters, und wenn beide Entwickler mit diesem Muster vertraut sind, dann können sie sich auf der gleichen Ebene unterhalten, ohne zu sehr ins Detail gehen zu müssen. Somit führt die Sprache, die durch bekannte und weitverbreitete Entwurfsmuster gebildet wird, eine höhere Ebene der Abstraktion ein, auf der wir mit der Komplexität anspruchsvoller Systeme besser umgehen können.

Entwurfsmuster für Software wurden bekannt durch ein 1995 veröffentlichtes Buch, das eine Reihe von Mustern, ihre Anwendbarkeit und ihre Vorteile beschreibt.⁵ Dieses Buch ist auch heute noch eines der wichtigsten Werke zum Thema Entwurfsmuster. Wir werden hier nicht versuchen, einen vollständigen Überblick über die darin beschriebenen Entwurfsmuster zu geben. Stattdessen werden wir im Folgenden nur eine kleine Auswahl dieser Muster diskutieren, um dem Leser einen Eindruck der Vorteile von Entwurfsmustern zu geben. Weitere Entwurfsmuster können dann in der entsprechenden Literatur nachgelesen werden.

⁵ *Design Patterns: Elements of Reusable Object-Oriented Software*, von Erich Gamma, Richard Helm, Ralph Johnson und John Vlissides, Addison-Wesley, 1995. Die exzellente deutsche Übersetzung von Dirk Riehle ist 1996 erschienen, ebenfalls bei Addison-Wesley.

15.7.1 Struktur eines Musters

Musterbeschreibungen werden üblicherweise in einer einheitlichen Form angegeben, die ein bestimmtes Minimum an Informationen garantiert. Eine Musterbeschreibung enthält nicht nur Informationen über die Struktur einiger Klassen, sondern auch über die Probleme, die dieses Muster angeht, sowie konkurrierende Einflüsse, die für oder gegen den Einsatz des Musters sprechen.

Die Beschreibung eines Musters enthält mindestens:

- einen **Namen**, der das Reden über das Muster erleichtert
- eine Beschreibung des **Problems**, das das Muster adressiert (häufig aufgeteilt in Abschnitte wie *Zweck*, *Motivation* und *Anwendbarkeit*)
- eine Beschreibung der **Lösung** (meist unterteilt in *Struktur*, *Teilnehmer* und *Partner*)
- eine Erläuterung der **Konsequenzen**, die sich aus dem Einsatz des Musters ergeben, mitsamt Ergebnissen und Vor- und Nachteilen

In den folgenden Abschnitten werden wir einige weitverbreitete Muster kurz vorstellen.

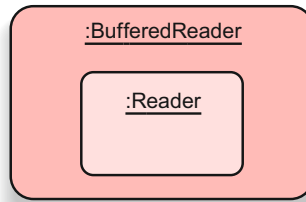
15.7.2 Dekorierer

Das Entwurfsmuster *Dekorierer* behandelt das Hinzufügen von Funktionalität zu einem bereits existierenden Objekt. Wir nehmen an, dass wir ein Objekt benötigen, das auf die gleichen Methodenaufrufe reagiert (also die gleiche Schnittstelle hat) wie unser Objekt, aber zusätzliches oder geändertes Verhalten zeigt. Möglicherweise wollen wir die Schnittstelle auch erweitern.

Eine Möglichkeit der Umsetzung ist über Vererbung. Eine Unterklasse könnte Implementierungen von Methoden überschreiben oder weitere Methoden hinzufügen. Aber die Verwendung von Vererbung ist eine statische Lösung: Einmal erzeugt, kann ein Objekt sein Verhalten nicht mehr ändern.

Eine dynamischere Lösung ist die Verwendung eines Dekorierer-Objekts. Ein Dekorierer ist ein Objekt, das ein anderes Objekt umschließt und anstelle des umschlossenen Objekts verwendet werden kann (es implementiert üblicherweise die gleiche Schnittstelle). Klienten kommunizieren dann mit dem Dekorierer statt direkt mit dem Original (ohne von dieser Ersetzung wissen zu müssen). Das Dekorierer-Objekt leitet die Aufrufe an das umschlossene Objekt weiter, kann aber auch zusätzliche Aktionen durchführen. Ein Beispiel liefert uns die Java-Bibliothek zur Ein-/Ausgabe. Dort ist ein **BufferedReader** ein Dekorierer für einen **Reader** (Abbildung 15.2). Ein **BufferedReader** implementiert die gleiche Schnittstelle und kann anstelle eines ungepufferten **Readers** verwendet werden, fügt aber zusätzliches Verhalten hinzu. Im Unterschied zur Vererbung kann ein Dekorierer auch für bereits existierende Objekte verwendet werden.

Abbildung 15.2
Struktur des Entwurfsmusters Dekorierer.



15.7.3 Singleton

In vielen Programmen gibt es Situationen, in denen es von einer bestimmten Klasse nur eine einzige Instanz geben sollte. In unserem *Zuul*-Spiel beispielsweise benötigen wir nur einen einzigen Parser. Wenn wir eine Softwareentwicklungsumgebung programmieren, dann benötigen wir möglicherweise nur einen einzelnen Compiler oder Debugger.

Das Entwurfsmuster *Singleton* garantiert, dass von einer Klasse nur eine einzige Instanz erzeugt wird, und ermöglicht einen einheitlichen Zugriff auf diese Instanz. In Java kann ein Singleton definiert werden, indem der Konstruktor privat deklariert wird. Dies garantiert, dass er nicht von außerhalb der Klasse aufgerufen werden kann und Klienten somit keine Instanzen erzeugen können. Wir erzeugen dann in der Singleton-Klasse selbst eine Instanz und ermöglichen den Zugriff auf sie (Listing 15.1 illustriert dies für die Klasse **Parser**).

Listing 15.1
Das Entwurfsmuster Singleton.

```
public class Parser
{
    private static Parser instanz = new Parser();

    public static Parser gibInstanz()
    {
        return instanz;
    }

    private Parser()
    {
        ...
    }
}
```

In dieser Umsetzung des Musters

- ist der Konstruktor privat, sodass Instanzen nur innerhalb der Klasse selbst erzeugt werden können. Dies muss im statischen Teil der Klasse passieren (bei der Initialisierung der statischen Felder oder in statischen Methoden), da sonst keine Instanzen existieren würden;
- wird ein privates, statisches Datenfeld definiert und initialisiert, das die (einzige) Instanz des Parsers hält;
- wird eine statische Methode **gibInstanz** definiert, die den Zugriff auf die eine Instanz ermöglicht.

Klienten des Singletons können diese statische Methode benutzen, um Zugriff auf das **Parser**-Objekt zu bekommen:

```
Parser parser = Parser.gibInstanz();
```

In der Tat bietet Java einen einfacheren Weg an, um die grundlegenden Funktionen eines Singletons zu erhalten – ein Aufzählungstyp mit einem einzigen Wert (Listing 15.3).

```
public enum Parser
{
    INSTANZ;

    // Parser-spezifische Methoden hier einfügen
}
```

Listing 15.2

Eine Singleton-Definition, die einen Java-Aufzählungstyp verwendet.

Auf die Singleton-Instanz würde dann über `Parser.INSTANZ` zugegriffen werden. Da beliebige Funktionalität in der Definition eines Aufzählungstyps ebenso wie in einer Klasse eingebaut werden kann, ist es dieser Ansatz wert, im Gegensatz zu einem handgestrickten Ansatz, in Java beachtet zu werden.

15.7.4 Fabrikmethode

Das Entwurfsmuster *Fabrikmethode* liefert die Schnittstelle für die Erzeugung von Objekten, überlässt jedoch Subklassen die Entscheidung darüber, von welchen konkreten Klassen Instanzen erzeugt werden. Typischerweise erwartet ein Klient eine Superklasse oder ein Interface vom dynamischen Typ des tatsächlichen Objekts, während die Fabrikmethode Spezialisierungen liefert.

Die Iteratoren der Sammlungsklassen (Subtypen von `Collection`) sind ein Beispiel für diese Technik. Wenn wir eine Variable vom Typ `Collection` haben, können wir sie nach einem Iterator fragen (mit der Methode `iterator`) und dann mit diesem arbeiten (Listing 15.3). Die Methode `iterator` ist in diesem Beispiel die Fabrikmethode.

```
public void verarbeite(Collection<Typ> c)
{
    Iterator<Typ> it = c.iterator();
    // ...
}
```

Listing 15.3

Die Benutzung einer Fabrikmethode.

Aus der Sicht des Klienten (im Quelltext in Listing 15.3) gehen wir mit Objekten vom Typ `Collection` und `Iterator` um. Tatsächlich ist der (dynamische) Typ der Sammlung jedoch möglicherweise `ArrayList`, wodurch der Aufruf von `iterator` ein Objekt vom Typ `ArrayListIterator` zurückliefert. Oder die Sammlung ist ein `HashSet` und `iterator` liefert einen `HashSetIterator`. Die Fabrikmethode wird in Subklassen spezialisiert, damit von dort spezialisierte Instanzen des „offiziellen“ Ergebnistyps geliefert werden.

Wir können dieses Muster sehr gut verwenden, um in unserer *Fuechse-und-Hasen-Simulation* aus Kapitel 12 die Klasse `Simulator` von den spezifischen Tierklassen zu entkoppeln. (Sie erinnern sich: In unserer Version der Simulation war der `Simulator` mit den Klassen `Fuchs` und `Hase` eng gekoppelt, weil er die Startpopulation erzeugt.)

Stattdessen können wir nun ein Interface `AkteurFabrik` einführen sowie Klassen, die dieses Interface für jeden Akteur implementieren (also etwa eine `FuchsFabrik` und eine `HasenFabrik`). Der `Simulator` würde einfach eine Sammlung von `AkteurFabrik`-Objekten halten und jede dieser Fabriken um die Erzeugung einer Anzahl von Akteuren bitten. Jede Fabrik würde einen anderen Typ von Akteur erzeugen, aber der `Simulator` würde mit den Fabriken nur über das Interface `AkteurFabrik` kommunizieren.

15.7.5 Beobachter

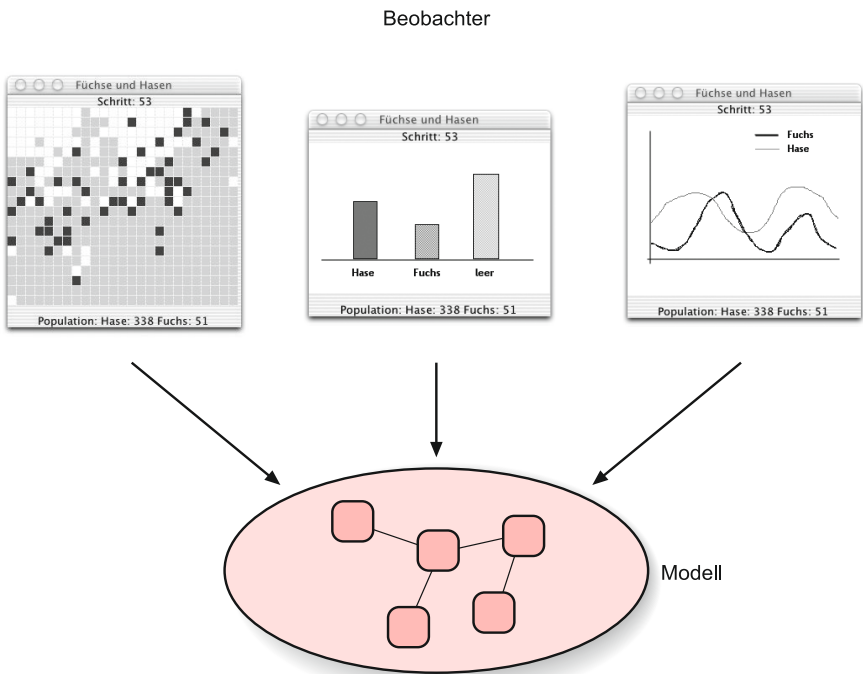
In der Diskussion vieler Projekte in diesem Buch haben wir versucht, das interne Modell einer Anwendung von seiner Darstellung auf dem Bildschirm (seiner Ansicht) zu trennen. Das *Beobachter*-Muster bietet eine Möglichkeit, diese Trennung zwischen Modell und Ansicht zu erreichen.

Allgemeiner gesprochen: Das Beobachter-Muster definiert eine 1-zu-*n*-Beziehung, sodass die Änderung des Zustands eines Objekts dazu führt, dass alle abhängigen Objekte benachrichtigt und automatisch aktualisiert werden. Es erreicht dies mit einer möglichst losen Kopplung zwischen den Beobachtern und dem beobachteten Objekt (dem sogenannten Subjekt).

Mit diesem Muster können nicht nur Modell und Ansicht entkoppelt werden, sondern auch mehrere Ansichten des gleichen Modells (entweder alternativ oder simultan) ermöglicht werden. Als Beispiel betrachten wir wieder unsere *Fuechse-und-Hasen*-Simulation.

In der Simulation wird die Population der Tiere auf dem Bildschirm in einem zwei-dimensionalen Feld animiert. Es gibt aber noch andere Möglichkeiten: Wir könnten auch die Darstellung als Graph der Populationszahlen auf einem Zeitstrahl vorziehen oder als bewegte Balkengrafik (Abbildung 15.3). Wir könnten auch alle Ansichten auf einmal sehen wollen.

Abbildung 15.3
Mehrere Ansichten für dasselbe Subjekt.



Für das Beobachter-Muster benutzen wir zwei Typen: **Observable** (englisch für beobachtbar) und **Observer**⁶ (englisch für Beobachter). Die beobachtbare Einheit (das

⁶ In dem Java-Paket `java.util` gibt es hierzu bereits zwei vordefinierte Typen: die Klasse **Observable** und das Interface **Observer** (das nur die Methode **update** aufweist).

Feld in unserer Simulation) erweitert die Klasse **Observable** und der Beobachter (Simulationsansicht) implementiert das Interface **Observer** (Abbildung 15.4).

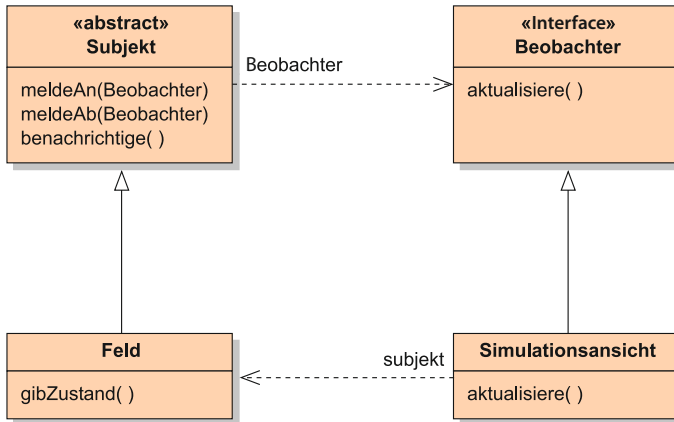


Abbildung 15.4
Struktur des Entwurfsmusters Beobachter.

Die Klasse **Observer** bietet Methoden an, mit denen sich Beobachter bei der beobachtbaren Einheit anmelden können. Sie garantiert, dass jedes Mal die Methode **update** an jedem Beobachter aufgerufen wird, wenn die beobachtete Einheit (das Feld) die geerbte Methode **notify** aufruft. Die tatsächlichen Beobachter (die Ansichten) können sich dann über den neuen, aktualisierten Zustand des Felds informieren und diesen erneut anzeigen.

Das Beobachter-Muster kann auch für andere Probleme als für die Trennung zwischen Modell und Ansicht benutzt werden. Es kann immer dann angewendet werden, wenn der Zustand eines oder mehrerer Objekte vom Zustand eines anderen Objekts abhängt.

15.7.6 Muster zusammengefasst

Die Diskussion über Entwurfsmuster und ihre Anwendungsmöglichkeiten geht weit über den Rahmen dieses Buches hinaus. Wir haben hier nur einen kurzen Eindruck davon vermittelt, was Entwurfsmuster sind, und eine informelle Beschreibung einiger weitverbreiteter Muster gegeben.

Wir hoffen jedoch, dass diese Diskussion die mögliche Richtung für weiteres Arbeiten aufzeigt. Sobald wir verstanden haben, wie wir gute Implementierungen einzelner Klassen erstellen können, können wir uns darauf konzentrieren, welche Arten von Klassen wir in unseren Anwendungen benötigen und wie diese kooperieren. Gute Lösungen sind oft nicht leicht zu finden, und Entwurfsmuster beschreiben Strukturen, die sich als nützlich für die Lösung immer wieder auftretender Probleme erwiesen haben. Sie unterstützen uns bei der Erstellung guter Klassenstrukturen.

Je erfahrener Sie als Softwareentwickler werden, desto mehr denken Sie über höhere Strukturen nach als über die Implementierung einzelner Methoden.

Übung 15.16 Drei zusätzliche häufig verwendete Entwurfsmuster sind *Zustand (state)*, *Strategie (strategy)* und *Besucher (visitor)*. Finden Sie Beschreibungen dieser Muster und benennen Sie jeweils mindestens einen Anwendungszusammenhang, in dem sie zum Einsatz kommen können.

Übung 15.17 In einer späten Phase eines Softwareprojekts stellen Sie fest, dass zwei Teams, die an unterschiedlichen Teilen der Anwendung gearbeitet haben, inkompatible Klassen programmiert haben. Die Schnittstelle einiger Klassen des einen Teams weichen leicht von den Schnittstellen ab, die das andere Team erwartet. Erklären Sie, wie das *Adapter*-Muster in dieser Situation helfen kann, ohne dass eine der bestehenden Klassen geändert werden muss.



Zusammenfassung

In diesem Kapitel haben wir uns auf eine höhere Ebene der Abstraktion bewegt, weg vom Entwurf einzelner Klassen (oder der Kooperation zwischen zwei Klassen) hin zum Entwurf einer gesamten Anwendung. Im Kern des Entwurfs eines objektorientierten Softwaresystems steht die Entscheidung, welche Klassen für seine Realisierung definiert werden und wie diese Klassen kooperieren sollen.

Einige Klassen sind recht offensichtlich und leicht zu entdecken. Wir haben die Verb/Substantiv-Methode bei einer textbasierten Problembeschreibung verwendet, um einen Startpunkt zu finden. Nach dem Identifizieren der Klassen können wir CRC-Karten und durchgespielte Szenarios benutzen, um die Abhängigkeiten und Details der Kommunikation zwischen den Klassen zu entwerfen und ihre Zuständigkeiten herauszuarbeiten. Weniger erfahrenen Entwicklern hilft es, wenn die Szenarios in einer Gruppe durchgespielt werden.

CRC-Karten können benutzt werden, um den Entwurf bis auf der Ebene der Methodennamen und ihrer Parameter zu verfeinern. Nachdem dies erfolgt ist, können Klassen mit Stubs in Java programmiert und ihre Schnittstellen dokumentiert werden.

Das Einhalten einer so organisierten Vorgehensweise dient mehreren Zwecken. Es stellt sicher, dass potenzielle Probleme mit frühen Entwurfsentscheidungen entdeckt werden können, bevor viel Aufwand in die Implementierung geflossen ist. Es ermöglicht auch, dass Programmierer parallel an mehreren Klassen arbeiten können, ohne dass bei der Implementierung der einen Klasse auf die Implementierung einer anderen gewartet werden muss.

Flexible, erweiterbare Klassenstrukturen sind nicht immer leicht zu entwerfen. Entwurfsmuster werden verwendet, um bewährte Strukturen zur Lösung verschiedener Problemklassen zu dokumentieren. Über das Studium von Entwurfsmustern kann ein Softwaretechniker sehr viel über gute Anwendungsstrukturen lernen und seine eigene Entwurfstechnik verbessern.

Je größer eine Aufgabenstellung ist, desto wichtiger ist eine gute Anwendungsstruktur. Je erfahrener ein Softwaretechniker wird, desto mehr Zeit verbringt er mit dem Entwurf von Anwendungsstrukturen statt allein mit dem Schreiben von Quelltext.

NEUE BEGRIFFE IN DIESEM KAPITEL

Analyse und Entwurf, Verb/Substantiv-Methode, CRC-Karte, Szenario, Geschäftsfall, Stub, Entwurfsmuster

Zusammenfassung der Konzepte

- **Verb/Substantiv** Die Klassen in einem System korrespondieren in etwa mit den Substantiven in der Systembeschreibung. Die Methoden korrespondieren mit den Verben.
- **Szenario** Szenarios (auch als „Geschäftsfälle“ oder unter dem englischen Begriff „use cases“ bekannt) können benutzt werden, um ein Verständnis von den Interaktionen in einem System zu bekommen.
- **Prototyping** Prototyping ist die Konstruktion unvollständiger Systeme, in denen einige Anteile simuliert werden. Es dient dazu, möglichst früh ein Verständnis von der Arbeitsweise eines Systems zu bekommen.
- **Entwurfsmuster** Ein Entwurfsmuster ist eine Beschreibung eines Entwurfsproblems, verbunden mit der Beschreibung einer kleinen Menge an Klassen und ihrer Interaktionsstruktur, mit deren Hilfe das Problem gelöst werden kann.



Übung 15.18 Angenommen Sie haben ein Schulverwaltungssystem für Ihre Schule, mit einer Klasse namens **Datenbank** (eine ziemlich wichtige Klasse), die Objekte vom Typ **Schueler** verwaltet. Zu jedem Schüler gehört eine Adresse, die in einem **Adresse**-Objekt gespeichert wird (d.h., jedes **Schueler**-Objekt hält eine Referenz auf ein **Adresse**-Objekt).

Jetzt möchten Sie von der **Datenbank**-Klasse aus auf die Straße, Stadt und Postleitzahl des Schülers zugreifen – Daten, für die die Klasse **Adresse** passende sondierende Methoden definiert. Für den Entwurf der Klasse **Schueler** haben Sie jetzt zwei Möglichkeiten:

Entweder implementieren Sie die Methoden **gibStrasse**, **gibStadt** und **gibPLZ** in der Klasse **Schueler**, die einfach den Methodenaufruf an das **Adresse**-Objekt weiterleiten und das Ergebnis zurückgeben, oder Sie implementieren in der Klasse **Schueler** eine Methode **gibAdresse**, die der **Datenbank** das vollständige **Adresse**-Objekt zurückliefert und das **Datenbank**-Objekt die Methoden des **Adresse**-Objekts direkt aufrufen lässt.

Welche dieser Möglichkeiten ist die bessere und warum? Erstellen Sie ein Klassendiagramm für jede Alternative und nennen Sie jeweils gute Gründe für die Lösung.



KAPITEL

16

Eine Fallstudie

Lernziele

Zentrale Konzepte in diesem Kapitel: Entwicklung einer kompletten Anwendung

Java-Konstrukte in diesem Kapitel: (In diesem Kapitel werden keine neuen Java-Konstrukte vorgestellt.)

In diesem Kapitel führen wir viele der objektorientierten Prinzipien zusammen, die wir in diesem Buch vorgestellt haben, indem wir eine ausführliche Fallstudie betrachten. Wir werden bei dieser Fallstudie von einer Problembeschreibung ausgehen, um dann über das Identifizieren der Klassen und den Entwurf zu einem iterativen Prozess von Implementierung und Test zu gelangen. Im Gegensatz zu den bisherigen Kapiteln wollen wir hier keine neuen Themen diskutieren, sondern den zweiten Teil des Buches wiederholen und vertiefen, insbesondere die Themen Vererbung, Abstraktionstechniken, Fehlerbehandlung und Anwendungsentwurf.

16.1 Die Fallstudie

In unserer Fallstudie werden wir ein Modell eines Taxiunternehmens entwickeln. Dieses Unternehmen untersucht gerade, ob es sein Einsatzgebiet in einen bisher nicht abgedeckten Teil der Stadt ausdehnen soll. Das Unternehmen betreibt einfache Taxis und Großraumtaxis. Einfache Taxis setzen ihre Fahrgäste erst am Zielort ab, bevor sie neue Fahrgäste aufnehmen. Großraumtaxis sammeln mehrere Fahrgäste an mehreren Abholpunkten auf und fahren diese während einer Fahrt zu verschiedenen Zielen (etwa indem sie Gäste in mehreren Hotels aufsammeln und diese zu verschiedenen Terminals am Flughafen fahren). Basierend auf den geschätzten Zahlen möglicher Fahrgäste im betrachteten Stadtteil möchte das Unternehmen herausfinden, ob eine Expansion profitabel sein kann und wie viele Taxis am neuen Standort für einen effektiven Betrieb notwendig wären.